

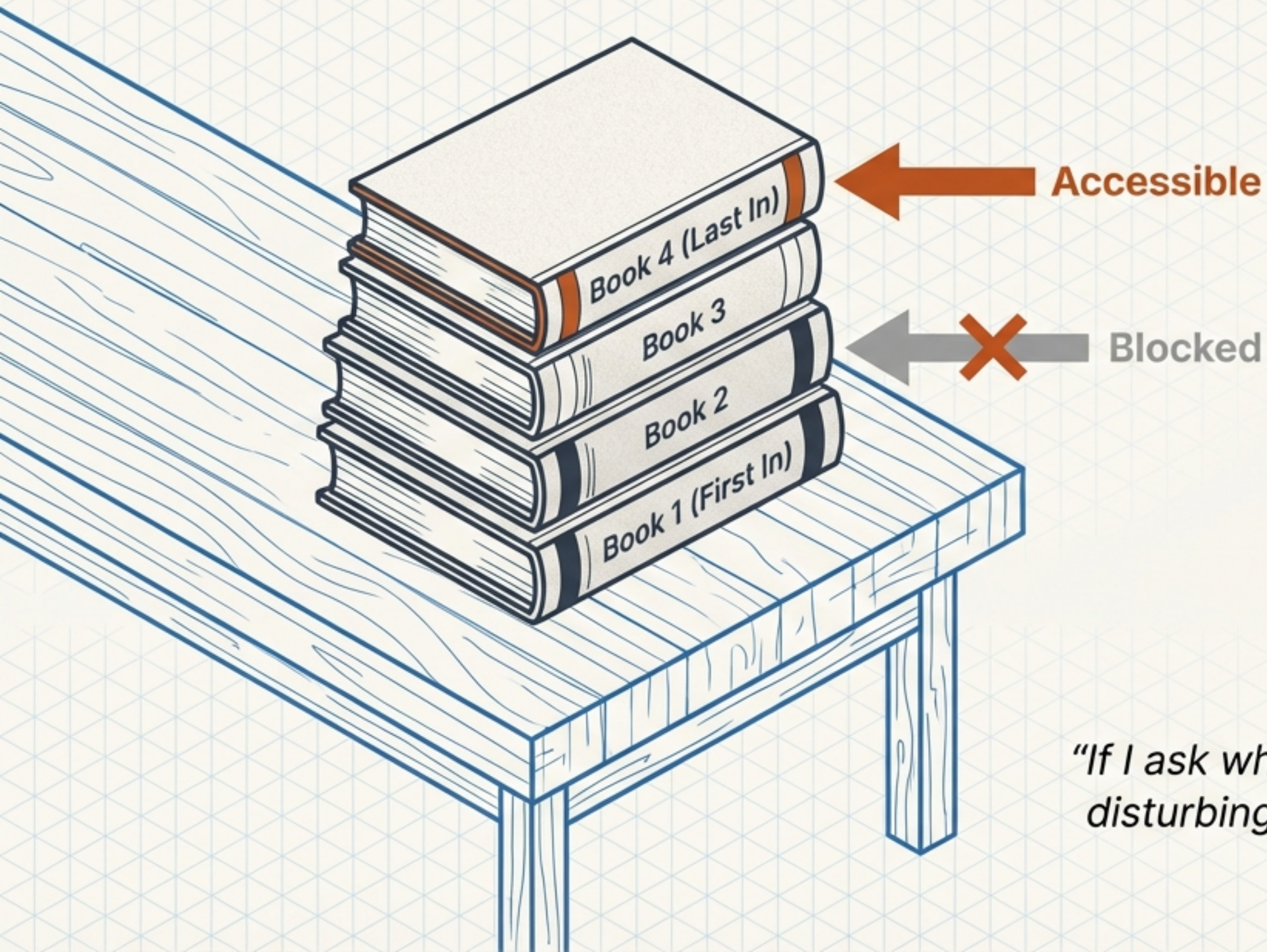
Mastering the Stack in Python

Data Structures & Algorithms Series | Chai aur Code



A comprehensive guide to understanding LIFO logic and implementing it from scratch.

The Mental Model: Last In, First Out (LIFO)



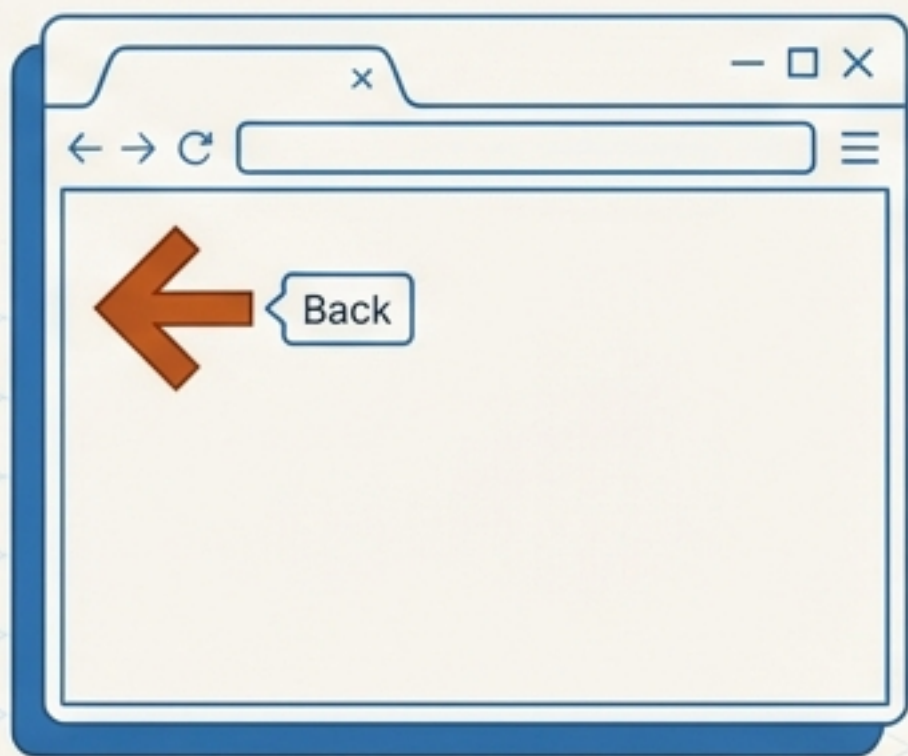
The Rule of LIFO:

The element inserted last is the first one accessed.

Just like a physical stack of books, you can easily pick up the top item, but you cannot pull out an item from the middle without disturbing the entire structure.

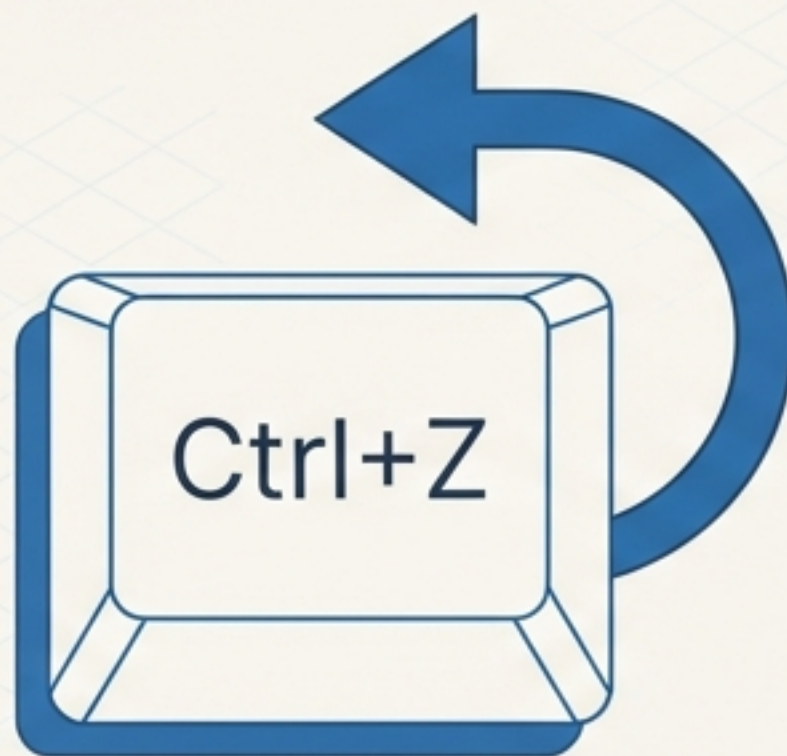
"If I ask which book you can pick up easily without disturbing the others? Obviously, the one on top."

Why We Need Stacks



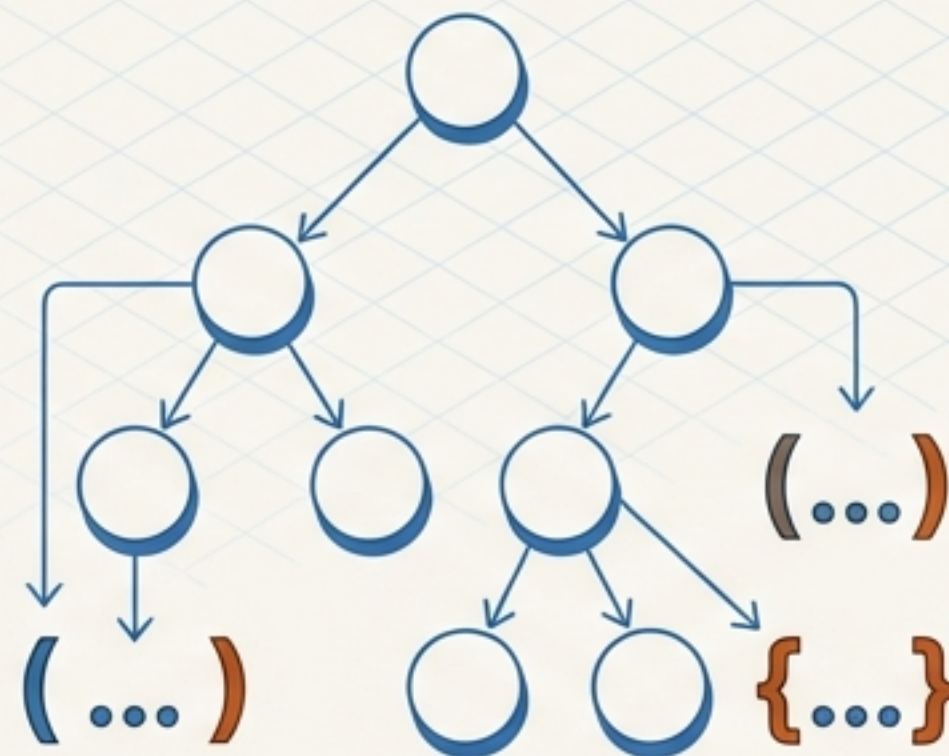
Browser History

Manages navigation. The 'Back' button pops the current page to return to the previous state.



Undo/Redo Operations

The backbone of text editors.
Every action is pushed; every
undo pops the last action.

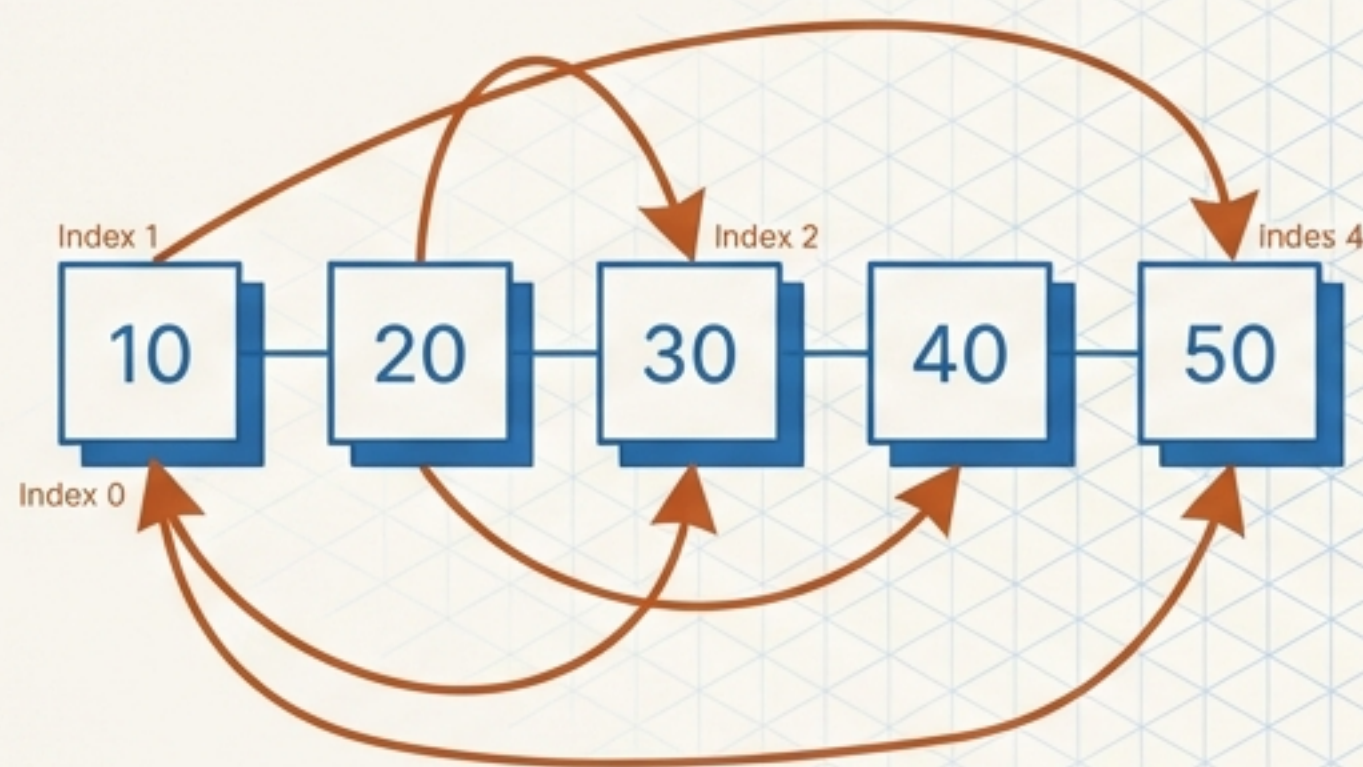


Recursion & Parsing

Manages memory in function calls and validates syntax (e.g., matching parentheses).

The Discipline of Structure

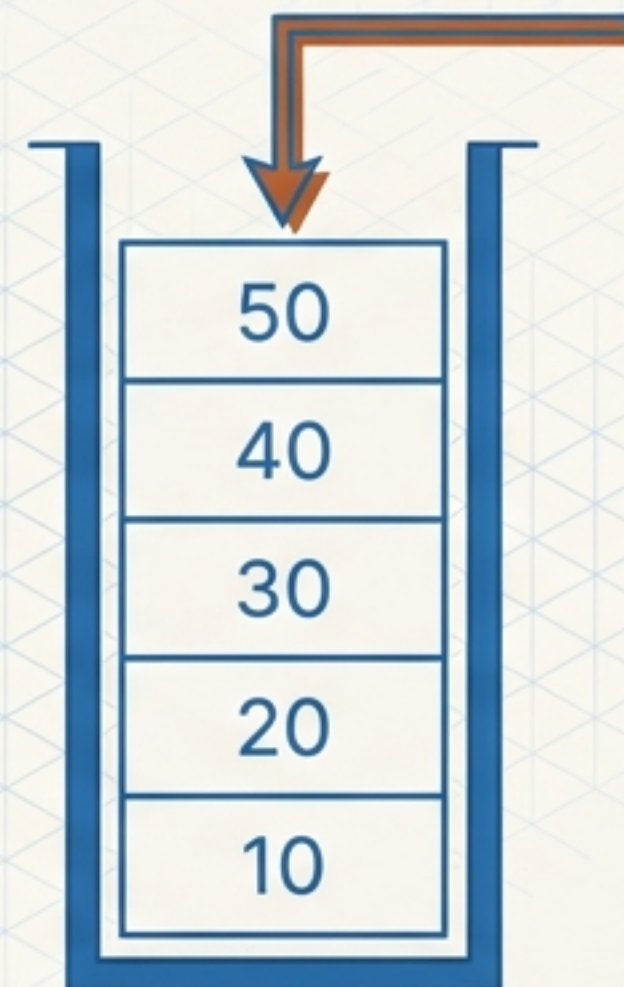
Python List (Array)



Random Access (Too much freedom)

Allows access to any element at any time.
Prone to unstructured modification.

The Stack



Controlled Access (Strict)

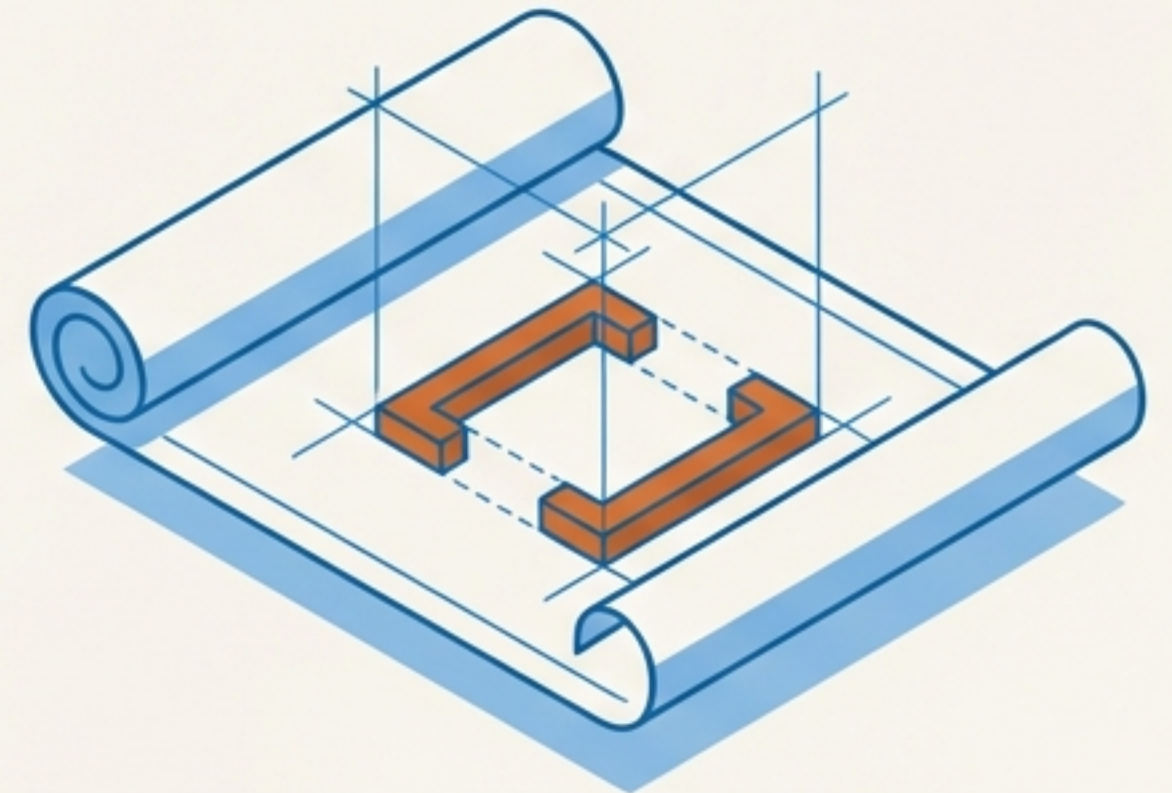
Constraint: Previous elements are 'untouched'.
We interact only with the latest addition.

The Foundation: Class Initialization

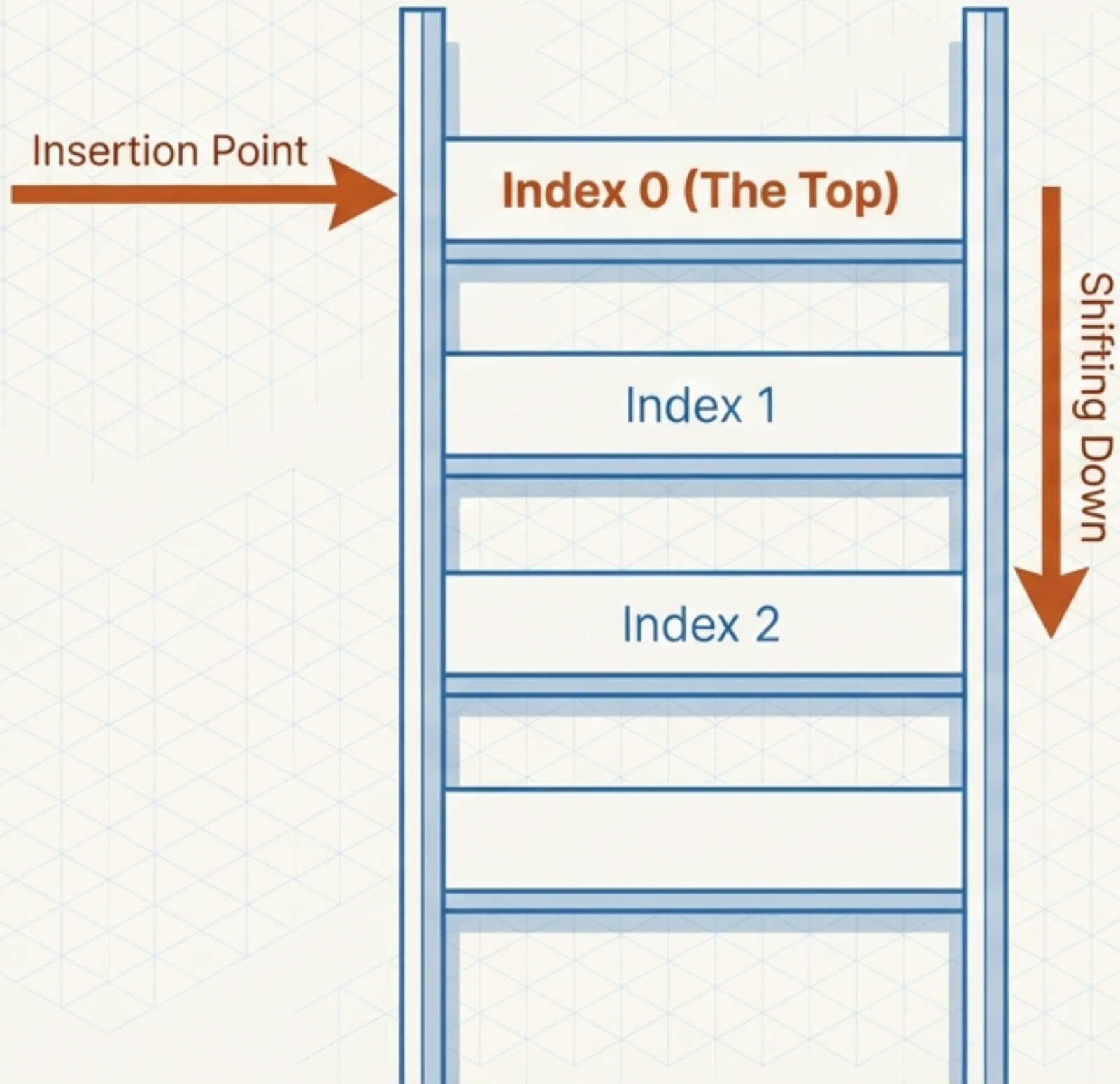
```
class Stack:  
    def __init__(self):  
        # We use a standard List as storage  
        self.s = []
```

****The Storage Mechanism****

We initialize a class `Stack` and create an empty list `self.s` to hold our data. While `self.s` is technically a list with full capabilities, our class methods will restrict how we use it.



The Strategy: Index 0 is “Top”



Design Choice:

- In this implementation, we treat **Index 0** as the mouth of the stack.
 1. New items are inserted at Index 0.
 2. Existing items are pushed down to higher indices.

Note: This differs from using `append()`, where the end of the list is the top.

Utility Methods: Length and Empty Check

```
def length(self):  
    return len(self.s)
```

```
def isEmpty(self):  
    return len(self.s) == 0
```

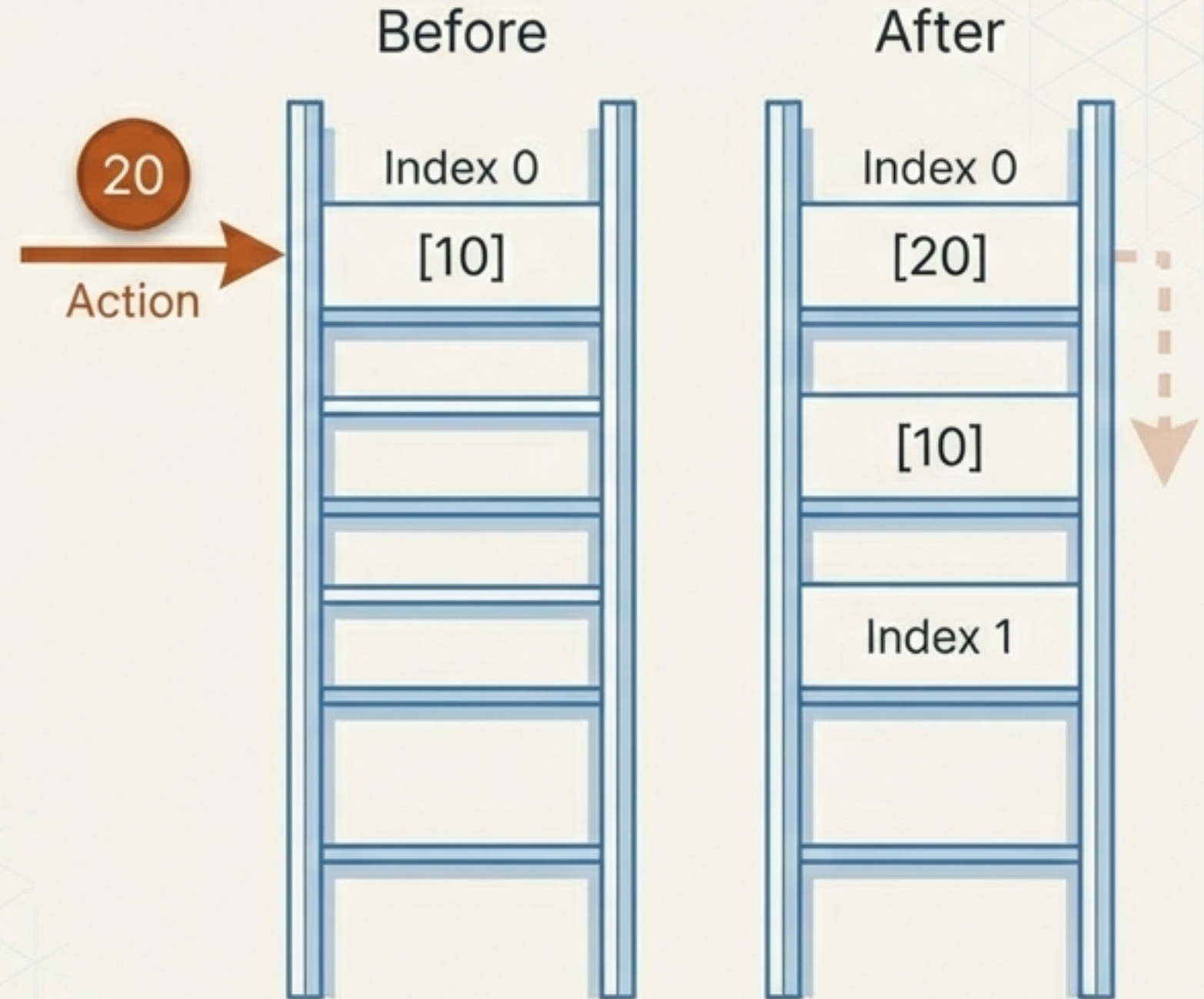
length(): Returns the total count of elements currently in the stack.

isEmpty(): A helper boolean check. Returns **True** if the stack size is 0. This is crucial for error handling before attempting to pop or peek.

Operation: PUSH (Insertion)

```
def push(self, val):  
    self.s.insert(0, val)
```

Using insert at index 0 ensures the new value takes the Top position.



Safety First: Handling Empty Stacks



The Rule: You cannot take money from an empty wallet.

Defensive Programming:

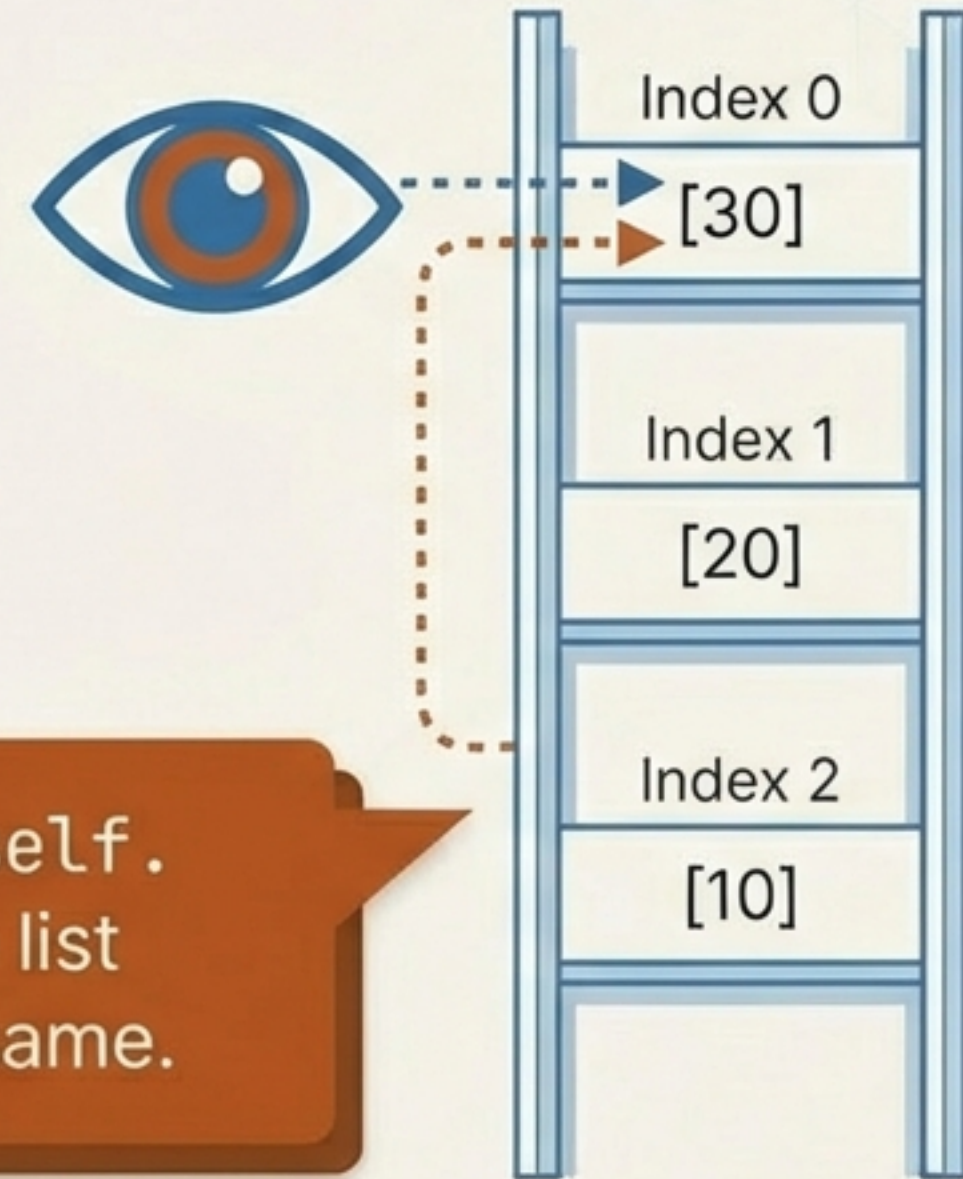
Before performing a `pop` or `peek` operation, we must verify the stack contains data.

If `isEmpty()` returns true, we explicitly raise an Exception to prevent silent failures.

```
if self.isEmpty():  
    raise Exception('Stack is Empty')
```


Operation: PEEK (Inspection)

```
def peek(self):  
    if self.isEmpty():  
        raise Exception('Stack is Empty')  
    return self.s[0]
```

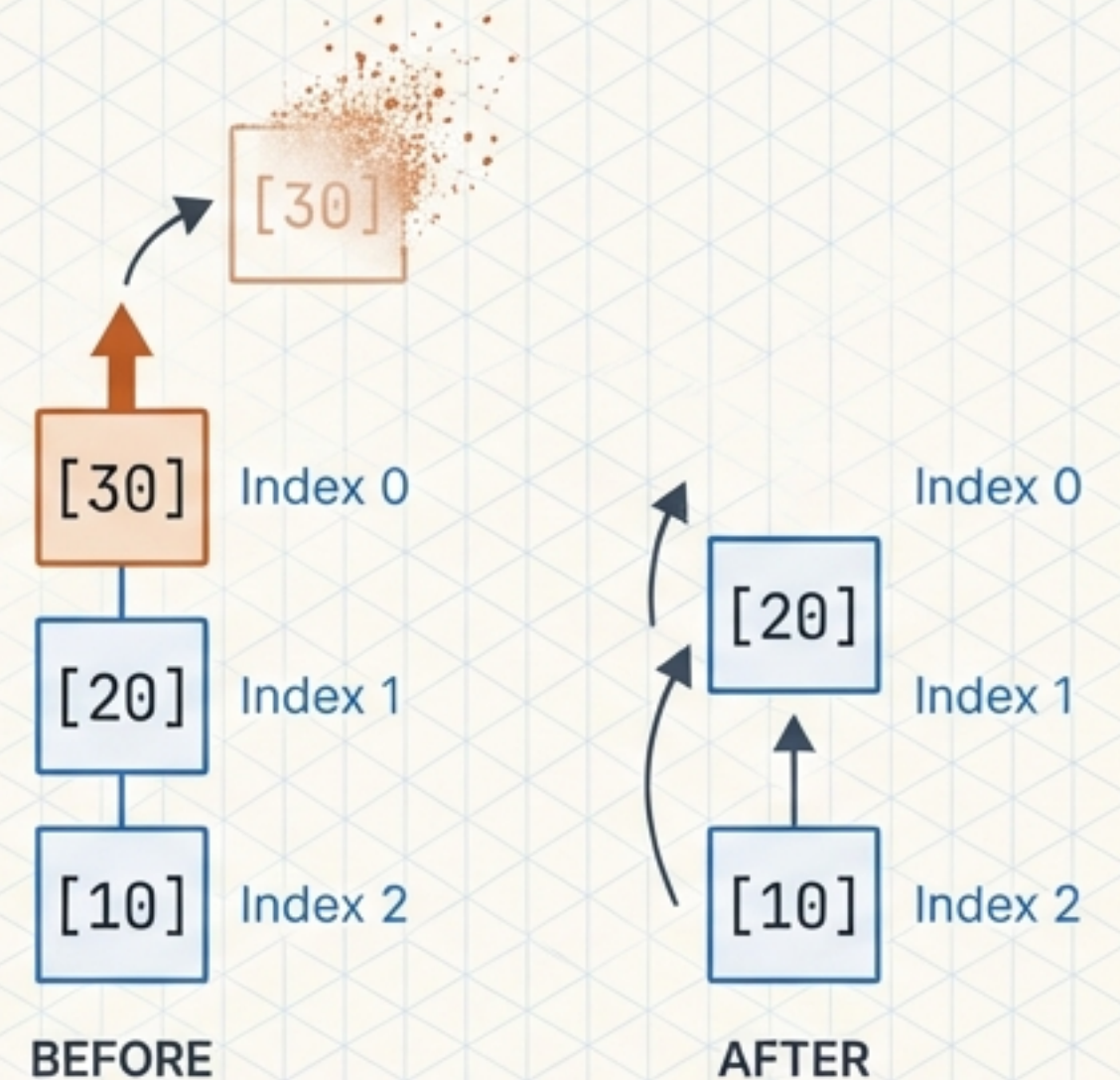


Non-Destructive: We access `self.s[0]` to view the value, but the list structure remains exactly the same.

Operation: POP (Deletion)

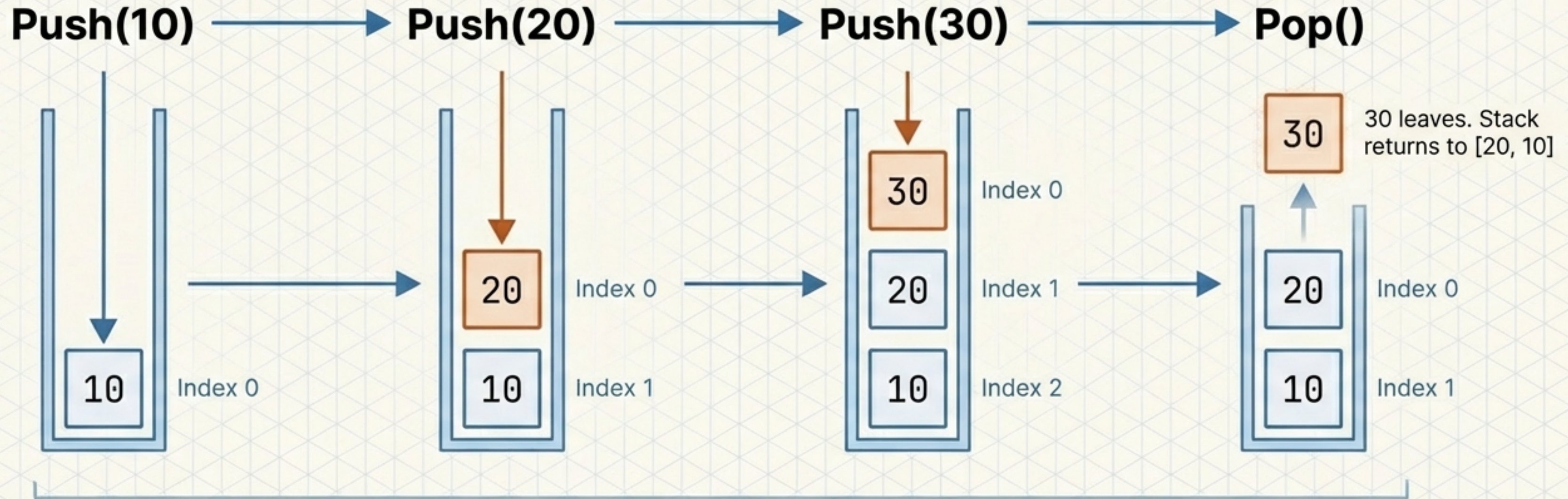
```
def pop(self):  
    if self.isEmpty():  
        raise Exception('Stack is Empty')  
    return self.s.pop(0)
```

30 is being lifted off
and fading away.



Using `pop(0)` removes the first item and shifts all remaining items up one index.

Simulating the Trace



The stack grows at Index 0. Deletion happens in reverse order of insertion. 10 entered first, but it will be the last to leave.

The Complete Class Implementation

```
class Stack:
    def __init__(self):
        self.s = []

    def length(self):
        return len(self.s)

    def isEmpty(self):
        return len(self.s) == 0

    def push(self, val):
        self.s.insert(0, val)

    def peek(self):
        if self.isEmpty():
            raise Exception('Stack is Empty')
        return self.s[0]

    def pop(self):
        if self.isEmpty():
            raise Exception('Stack is Empty')
        return self.s.pop(0)
```


Challenge: The Alternative Approach

Current Method

- Top is **Index 0**
- Uses ``insert(0, val)``
- Uses ``pop(0)``
- **Note:** Requires shifting elements.

Homework Task

- Top is **The End**
- Use ``append(val)``
- Use ``pop()`` (no index)
- **Note:** Generally more efficient ($O(1)$).

Your Mission: Refactor the class to add elements to the **end of the list** instead of the beginning.

Final Advice: Learn and Move On

Don't get stuck here.

- Stacks are foundational but simple.
- Watch the video, write the code, solve 1-2 problems.
- Move on to the next topic.

Share your progress on Twitter/X with **#ChaiAurDSA**

